# 写给设计师的 DOM 科普：AI 时代，你的图层树为什么突然重要了（2026-09-17）

## 写给设计师的 DOM 科普：AI 时代，你的"图层树"为什么突然重要了

> 面向 UED 设计师 · 2026-09-17
> 30 秒版：**DOM 就是"页面的图层树"。AI 读页面、AI 走查、D2C 生成代码，读的都是它。你的 Figma 结构有多清晰，AI 理解你的设计就有多准确。**

---

### 一、DOM 是什么？—— 就是页面的"图层树"

打开 Figma 左侧面板，你看到的是图层树（Layer Tree）：Frame 套 Group，Group 里有 Component 和 Text，一层套一层。

浏览器里发生的事一模一样：开发者写的 HTML 代码，浏览器会把它解析成一棵树，叫 **DOM（Document Object Model，文档对象模型）**。你看到的每一个页面，都是浏览器"渲染"这棵树的结果。

**设计稿和页面的结构是一一对应的：**

| Figma 里的概念 | DOM 里的对应物 | 说明 |
| --- | --- | --- |
| Frame / Group（画板/编组） | `<div>` / `<section>`（容器节点） | 装东西的盒子，负责布局 |
| Auto Layout（自动布局） | Flexbox（弹性布局） | 间距、对齐、自适应的规则 |
| Component（组件） | 组件类 / 自定义元素 | 定义一次，到处复用 |
| Instance（实例） | 组件实例 | 改主组件，全部实例跟着变 |
| Text（文本图层） | 文本节点 | 纯文字内容 |
| 图层命名 | class / id 属性 | 给节点起名字，方便找到它 |
| Variants（变体） | 状态切换（如 hover、选中） | 同一个组件的不同状态 |
| 图层样式（填充/描边/圆角） | CSS 样式 | 长什么样的描述 |

所以当你听到开发说"这个 DOM 结构有问题"，翻译成设计师的话就是：**"这个页面的图层组织方式有问题"**——比如该成组的地方没成组、一个盒子套了八层没必要的盒子。

---

## 二、为什么设计师以前就要懂一点，现在必须懂

**以前的理由（还原度）**：懂 DOM 你才知道为什么"这个间距还原不了"、"为什么这个效果在手机上错位"——很多走查问题本质是设计结构和 DOM 结构没对上。

**现在的理由（AI）**：AI 工具正在大规模地"读"和"写" DOM——

- **D2C**（设计稿转代码）：AI 读你的 Figma 节点树，生成 DOM
- **AI 走查**：AI 读页面的 DOM，和设计稿标注比对，检查还原度
- **AI 浏览器/Agent**：AI 读 DOM 理解页面结构，决定点哪里、填什么（比如自动化测试、智能客服帮用户操作页面）
- **页面理解**：搜索引擎、AI 助手读 DOM 提取页面内容

**一句话：DOM 是设计、开发、AI 三方共同使用的"页面通用语言"。**

---

## 三、DOM 喂给 LLM，到底喂的是什么？

这是最容易被误解的地方。**不是把原始 HTML 整个扔给 AI**——一个普通首页的 DOM 可能有几万个节点，换算成 AI 的"食量"（token）远超它的胃口，而且 90% 是噪声（脚本、样式、无意义的嵌套包装）。

实际做法是**先瘦身，再投喂**，通常四步：

### 第 1 步：清洗（扔掉噪声）

去掉 `<script>`（逻辑代码）、`<style>`（样式代码）、注释、不可见的元素。这些对"理解页面"没用，纯占地方。

### 第 2 步：压缩（拍平结构）

合并无意义的多层嵌套，只保留**有内容的**（文本、图片）和**可交互的**（按钮、输入框、链接）节点。

### 第 3 步：序列化（树 → 文本）

LLM 读的是文字，不是图。把树结构转成带缩进和编号的文本清单，每个可交互元素给一个编号，方便 AI 引用（"点 16 号按钮"）。

### 第 4 步：语义化（保留"意思"）

优先保留标签类型、aria 属性、文本内容——AI 靠这些判断"这是个按钮"还是"这只是个长得像按钮的色块"。

**一个直观例子**。页面上一个酒店卡片，在浏览器里原始 DOM 可能是几百行嵌套代码；瘦身喂给 AI 之后，变成这样：

代码块

```
[12] <div 酒店卡片>
  [13] <图片 描述="成都春川路希尔顿酒店外观">
  [14] <标题>成都春川路希尔顿酒店</标题>
  [15] <文本>¥586 起</文本>
  [16] <按钮>查看详情</按钮>
</div>
```

AI 拿到这个，立刻就懂：这是一张酒店卡片，有图、有名字、有价格、有个按钮可以点。

> 进阶一点的做法：用**无障碍树（Accessibility Tree）**替代 DOM——它本来就是为"屏幕阅读器理解页面"设计的精简语义结构，天然适合喂给 AI。以及**截图 + DOM 双通道**：截图负责视觉理解，DOM 负责结构理解，两个一起喂。

---

## 四、AI 拿到 DOM 之后，拿它干什么？

| 场景 | AI 用 DOM 做什么 | 和设计师的关系 |
| --- | --- | --- |
| **D2C（设计稿→代码）** | 读 Figma 节点树 → 生成 DOM 结构 | 你的图层结构就是输入，垃圾进垃圾出 |
| **AI 走查/还原度检查** | 读页面 DOM 的实际值（间距、字号、颜色）和设计标注比对 | 结构对得上，AI 才查得准 |
| **AI 浏览器操作** | 按编号定位元素，执行点击、输入 | 页面结构清晰，Agent 才不会"点错" |
| **内容理解/搜索** | 提取正文、价格、标题等关键信息 | 语义化做得好，AI 才抓得对 |

---

## 五、对设计工作的三个直接影响

### 1. Figma 图层命名和组件化 = AI 时代的"设计可读性"

以前图层随便命名（"编组 237""矩形 12"），痛苦的只是接手的同事；现在 D2C 工具读你的设计稿，**图层名、组件名就是它理解设计意图的唯一线索**。命名即文档，组件化即机器可读。

### 2. 用组件，而不是"画一个长得像的东西"

设计稿里用矩形+文字拼一个"看起来像按钮"的图形，D2C 转出来就是一个没有语义的色块 `<div>`——AI 不知道它能点，无障碍工具不知道它是按钮，搜索引擎不知道它是入口。**用组件库里真正的 Button 组件，等于告诉全世界（包括 AI）：这是一个按钮。**

### 3. 结构规范是 AI 时代的设计基础设施

SDUI、组件库智能化、设计交付规则——这些不是流程负担，而是在定义"什么样的页面结构是 AI 能稳定理解的"。**规范越结构化，AI 工具链（D2C、走查、Agent）的效果就越好**，这就是"设计知识的数字化"。

---

## 六、速查表（可以截图收藏）

| 你听到的话 | 设计师翻译 |
| --- | --- |
| DOM | 页面的图层树 |
| DOM 节点 | 一个图层（容器/文本/图片） |
| class / id | 图层的名字 |
| 语义化 | 用对组件，让机器看懂"这是什么" |
| DOM 瘦身 | 扔掉图层树里的噪声，只留有用的给 AI 看 |
| Accessibility Tree | 给屏幕阅读器/AI 看的"精简版图层树" |
| D2C | AI 读你的 Figma 图层树，生成页面代码 |

---

## 一句话总结

**设计稿是"设计的 DOM"，页面是"浏览器的 DOM"——AI 时代，它们之间的翻译质量，决定了 AI 能在多大程度上理解、检查和生成你的设计。把图层树收拾干净，就是在给 AI 铺 rails。**